



HTTPS/TLS, Contraseñas y Gestión de Secretos

Taller de Seguridad Web – OWASP Top 10

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

Agenda del día

Sesión 1 (2h)

Bloque	Tema
Repaso	Warm-up día 4
T10	HTTPS, TLS y cifrado en tránsito
Lab	Inspección de certificados
T11	Contraseñas — almacenamiento seguro
Lab	Hashing vs cracking

Sesión 2 (2h)

Bloque	Tema
T12	Gestión de secretos en repos
Lab	Detectar secretos con truffleHog
T12	Azure Key Vault y .env
Lab	Mover secretos a variables de entorno

Objetivos del día

Al terminar hoy serás capaz de:

1.  Explicar el TLS handshake y verificar certificados con SSL Labs
2.  Entender por qué MD5/SHA son inútiles para passwords y bcrypt/Argon2 son seguros
3.  Detectar secretos filtrados en repositorios Git con truffleHog
4.  Gestionar secretos correctamente con `.env` y Key Vault

💡 **Hoy es día de DEFENSA.** Los ataques son menos espectaculares pero las defensas son **fundamentales**.

OWASP Top 10:2025 — Progreso

#	Categoría	Día(s)	Estado
A01	Broken Access Control	4	✓
A02	Security Misconfiguration	6-7	
A03	Software Supply Chain Failures	6, 8	
A04	Cryptographic Failures (TLS+Passwords+Secrets)	5	🔥 HOY
A05	Injection	1-3	✓
A06	Insecure Design	4, 6	½
A07	Authentication Failures	3-4	✓
A08	Software/Data Integrity Failures	8	
A09	Security Logging & Alerting	8	
A10	Mishandling of Exceptional Conditions	3	✓

4 categorías completadas · **Hoy: A04 — criptografía y secretos**

El problema de hoy en 3 imágenes

HTTP sin cifrar:

```
WiFi pública:  
username=alice  
password=password123  
→ VISIBLE para  
cualquiera
```

Contraseñas en BD:

```
SELECT * FROM users;  
-- alice | password123  
-- bob   | qwerty  
→ LEGIBLE si accedes  
a la BD
```

Secretos en Git:

```
const JWT_SECRET = 'secret';  
// → git log lo muestra  
// → bots lo encuentran  
// → exploit en <1h
```

💀 **Datos sin cifrar = brecha inevitable.** Solo es cuestión de cuándo, no de si.

T10: HTTPS, TLS y Cifrado en Tránsito

Si no cifras el canal, todo lo demás es inútil

HTTP vs HTTPS – Lo que ve un atacante en la red

HTTP (sin cifrar):

```
POST /login
username=alice
password=password123
```

- Visible con Wireshark
- Cualquiera en la WiFi puede leerlo

HTTPS (con TLS):

```
17 03 03 00 1a 8f 2c b1
a4 5e 9d 7f 3c 8b 2a 6d
f1 c4 ...
```

- Cifrado AES-256
- Solo el servidor puede descifrarlo

TLS Handshake — Los 5 pasos



Cipher suite actual: `TLS_AES_256_GCM_SHA384`

- TLS 1.3 + cifrado AES-256 en modo GCM + hash SHA-384

HSTS — Forzar HTTPS siempre

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```

Sin HSTS	Con HSTS
Usuario escribe <code>http://</code> → atacante intercepta la redirección	Navegador nunca envía HTTP (convierte a HTTPS localmente)
SSL stripping posible	SSL stripping imposible

✅ **HSTS preload:** Incluir el dominio en la lista hardcodeada de Chrome/Firefox → HTTPS desde la primera visita.

Errores comunes en TLS

Error	Riesgo	Solución
TLS 1.0/1.1 habilitado	Ataques POODLE, BEAST	Solo TLS 1.2+ (idealmente 1.3)
Certificado expirado	Usuarios ignoran warning → MITM	Renovación automática (Let's Encrypt)
Cipher suites débiles (RC4, 3DES)	Descifrado offline	Solo AES-128/256-GCM
Sin HSTS	SSL stripping	Añadir header + preload
Mixed content (HTTP en página HTTPS)	Recursos sin cifrar	Auditar con DevTools

Lab T10 — Verificar el certificado TLS

1. Navegar a `https://vulnapp-owasp-01.azurewebsites.net`
2. Clic en el candado → Ver certificado:
 - ¿Quién lo emitió? (DigiCert / Let's Encrypt / Microsoft)
 - ¿Cuándo expira?
 - ¿Qué cipher suite se usa?
3. Ir a ssllabs.com/ssltest → Analizar la URL
 - ¿Qué nota obtiene? (A, B, C, F?)
 - ¿TLS 1.3 soportado?

 **Herramientas alternativas:** `testssl.sh` (CLI), securityheaders.com

T11: Contraseñas — Almacenamiento Seguro

De texto plano a bcrypt/Argon2

La evolución del almacenamiento de passwords

Método	Velocidad cracking	Seguridad
Texto plano	Instantáneo (leer)	XXX
MD5	~10 mil millones/seg (GPU)	XX
SHA-256	~5 mil millones/seg (GPU)	X
bcrypt(12)	~3 hashes/seg	✓✓
Argon2id	Configurable (memoria + CPU)	✓✓✓

💀 **VulnApp almacena passwords en TEXTO PLANO:** `alice | password123`. Si la BD se filtra → todas las cuentas comprometidas al instante.

Demostración: registra un usuario en `/auth/register` y luego consulta `/debug` para ver que el password está en claro.

¿Por qué MD5/SHA son inútiles para passwords?

```
MD5("password123") → 482c811da5d5b4bc6d497ffa98491e38
```

Problema 1 — Velocidad: Una GPU moderna calcula 10 mil millones de MD5/seg.

```
Diccionario rockyou.txt (14M passwords): crackeado en < 2 segundos
```

Problema 2 — Rainbow tables: Tablas precomputadas con millones de hashes.

```
Google: "482c811da5d5b4bc6d497ffa98491e38" → password123
```

Problema 3 — Sin salt: Dos usuarios con la misma password → mismo hash.

bcrypt — La solución estándar

```
import bcrypt from 'bcryptjs';

// Al registrar usuario:
const hash = await bcrypt.hash('password123', 12);
// → $2b$12$LJ3m4yB9hQz8KgNK... (60 chars, incluye salt)

// Al hacer login:
const valid = await bcrypt.compare(inputPassword, user.password_hash);
if (!valid) return res.status(401).json({ error: 'Credenciales inválidas' });
```

¿Por qué funciona?

- **Salt aleatorio** → mismo password ≠ mismo hash
- **Cost factor (12)** → ~0.3 seg/hash → cracking infeasible
- **95.000 años** para crackear un password decente con bcrypt(12)

Argon2id — El estado del arte

```
import argon2 from 'argon2';

const hash = await argon2.hash(password, {
  type: argon2.argon2id,    // Resistente a side-channel + GPU
  memoryCost: 65536,       // 64 MB de RAM por hash
  timeCost: 3,             // 3 iteraciones
  parallelism: 4            // 4 threads
});

const valid = await argon2.verify(hash, inputPassword);
```

Ventaja sobre bcrypt: requiere **mucha RAM** → GPUs/ASICs no pueden paralelizar eficientemente.

NIST 800-63B — Políticas de contraseñas modernas

✓ Sí hacer	✗ NO hacer
Mínimo 8 caracteres	Forzar rotación periódica
Blacklist top 10K passwords comunes	Exigir símbolos obligatorios
Permitir hasta 64+ caracteres	Limitar a 12-16 chars
Habilitar MFA	Solo password como factor
Verificar contra breaches (HaveIBeenPwned)	Preguntas de seguridad

Lab T11 — Hashing vs Cracking

Ejercicio 1 — Cracking instantáneo:

1. Ir a crackstation.net
2. Pegar: `482c811da5d5b4bc6d497ffa98491e38` (MD5 de "password123")
3. ¿Cuánto tarda? → Instantáneo

Ejercicio 2 — bcrypt es invencible:

1. Hash bcrypt: `$2b$12$LJ3m4yB9hQz8KgNKG1dC70xHVb.YNdRdRfU2YChXhqLHKqUE5IiAe`
2. Intentar en CrackStation → **no lo encuentra**
3. ¿Por qué? El salt hace que las rainbow tables sean inútiles

 **Conclusión:** texto plano y MD5 se crackean en segundos. bcrypt/Argon2 son prácticamente imposibles.

T12: Gestión de Secretos

Secretos en repos, .env y Key Vault

El problema – Secretos filtrados en GitHub

Estadísticas 2023:

- 12.8M secretos filtrados en GitHub público
- Tiempo medio de explotación: < **1 hora**
- Coste medio de un incidente: **\$1.2M**

Bots automatizados:

Escanean GitHub en tiempo real:

- AWS keys → minado cripto
- JWT secrets → impersonación
- DB passwords → exfiltración
- Stripe keys → fraude

💀 Si un secreto llega a Git, considéralo **comprometido**. `git filter-branch` no lo elimina de forks ni caches.

Herramientas de detección

```
# truffleHog - busca secretos por entropía y patrones  
npx trufflehog filesystem . --no-verification
```

```
# gitleaks - análisis de repos Git  
gitleaks detect --source . --report-path leaks.json
```

```
# git-secrets (AWS) - pre-commit hook  
git secrets --install  
git secrets --register-aws
```

Resultado típico en VulnApp:

```
Found: JWT_SECRET = 'secret' in src/routes/auth.ts:7  
Found: Hardcoded passwords in src/db/database.ts:57-59  
Found: Debug endpoint without auth in src/routes/debug.ts
```

Capas de protección para secretos

Capa	Herramienta	Cuándo actúa
1. Developer	<code>.env</code> + <code>.gitignore</code>	Al escribir código
2. Pre-commit	git-secrets, husky hook	Antes del commit
3. CI/CD	gitleaks, truffleHog en pipeline	Al hacer push
4. Runtime	Azure Key Vault / AWS Secrets Manager	En producción
5. Rotación	Automática (Key Vault policy)	Periódicamente

Azure Key Vault — Secretos fuera del código

```
import { SecretClient } from '@azure/keyvault-secrets';
import { DefaultAzureCredential } from '@azure/identity';

// La app se autentica con Managed Identity (sin passwords!)
const client = new SecretClient(
  'https://my-vault.vault.azure.net',
  new DefaultAzureCredential()
);

// Obtener el secreto en runtime
const secret = await client.getSecret('jwt-secret');
const JWT_SECRET = secret.value!;
```

✓ **Ventajas:** rotación automática, auditoría de accesos, cifrado at-rest, control RBAC.

Regla de emergencia — Secreto filtrado

1. ⚡ REVOCAR inmediatamente (invalidar la clave/token)
 2. 🔄 ROTAR – generar nueva clave y desplegar
 3. 🔍 INVESTIGAR – ¿alguien la usó? (logs)
 4. 🛡️ PREVENIR – añadir pre-commit hook
- ⚠️ git filter-branch / BFG Repo-Cleaner eliminan del repo local pero NO de:
- Forks existentes
 - Caches de CI/CD
 - Clones de otros devs
 - Archive.org / Google cache

Lab T12 — Detectar y mover secretos

Parte 1 — Encontrar secretos:

1. Buscar en VulnApp: `grep -r "secret\|password\|key" src/`
2. ¿Cuántos secretos hardcodeados encuentras?
3. Visitar `/debug` → ¿revela variables de entorno?

Parte 2 — Plaintext passwords (demo en vivo):

```
# Registrar un usuario nuevo:
curl -s -X POST https://vulnapp-owasp-01.azurewebsites.net/auth/register \
  -H "Content-Type: application/json" \
  -d '{"username":"test_plain","password":"SuperSecreto123!","email":"t@t.com"}'

# Verificar que está en texto plano:
curl -s https://vulnapp-owasp-01.azurewebsites.net/debug
# → Buscar el password en la respuesta - ¿aparece en claro?
```

Parte 3 — Crear `.env`:

```
# Crear archivo .env
JWT_SECRET=mi-clave-super-segura-de-256-bits-aqui
PORT=3000
DB_PATH=./vuln.db
```

La solución completa — capas de cifrado

Capa	Problema	Solución
En tránsito	HTTP texto plano	HTTPS + TLS 1.3 + HSTS
En reposo (BD)	Passwords legibles	bcrypt(12) / Argon2id
En código	Secretos hardcodeados	<code>.env</code> + Key Vault + rotación

TLS handshake	→ cifra el canal (nadie lee el tráfico)
bcrypt(12)	→ hace inviable el cracking (95.000 años)
Key Vault	→ secretos fuera del código con rotación automática

Resumen del día

Tema	Problema	Solución	Herramienta
TLS	HTTP sin cifrar	HTTPS + TLS 1.3 + HSTS	SSL Labs
Passwords	Texto plano en BD	bcrypt(12) / Argon2id	bcryptjs
Secretos	Hardcodeados en código	<code>.env</code> + Key Vault	truffleHog, gitleaks

🏆 **Logro del día:** Entendéis las 3 capas de cifrado (tránsito, reposo, código) y sabéis implementar cada una.